Part 2: Indexing and Evaluation

Group members: Júlia Hernan (u233681), Alan Le Roux (u233515), Alba Miró (u233727)

GitHub repository: https://github.com/juliahupf/RIAW.git

1. Introduction

This report describes the second phase of our search engine. Starting from the 28,080 documents pre-processed in Part 1, we build an inverted index, answer conjunctive (AND) queries and rank the results with TF-IDF. Then we implement seven evaluation metrics, apply them to the two validation queries and to five test queries of our own, for which we create the relevance judgments ourselves. The last sections analyse what the metrics say about the system and which problems it has.

The search code is in `myapp/search/algorithms.py`, the metrics in `myapp/search/evaluation.py`, and all the experiments, tables and figures in `project_progress/part_2/part2.ipynb`. Section 5 explains how to reproduce them.

2. Indexing

2.1 Query pre-processing

A query must be processed exactly like the documents: if a user writes "running" and the index stores the stem `run`, there is no match. The function `build_terms(text)` therefore repeats the pipeline of Part 1: case folding, URL removal, punctuation replaced by spaces, splitting on whitespace, NLTK English stop-word removal, removal of single letters (digits are kept) and Porter stemming. For example, "men running shoes" becomes `['men', 'run', 'shoe']`. The function is copied from the Part 1 notebook because code inside a notebook cannot be imported from a Python module.

2.2 Inverted index and conjunctive queries (Task 1)

`create_inverted_index(docs, field)` builds the index on `all_terms`, the field that Part 1 created by merging title, description, brand, category, sub_category and product_details. The statement suggests storing, for each term, the list of documents that contain it. We store that list as the keys of a dictionary, and the number of occurrences (term frequency) as the values, because TF-IDF needs it:

```
index = {
    "kurta":  {"ETHFPDUMHA2BTSZH": 2, "KTAFZ...": 1, ...},
    "cotton": {"TKPFCZ9EA7H5FYZH": 2, ...},
    ...
}
```

The resulting vocabulary has **6,768 terms**. `conjunctive_match(query_terms, index)` implements the AND semantics required by the statement: a document is returned only if it contains every query term. It intersects the posting lists of the query terms, starting with the shortest one, because the result can never be larger than that list. If a query term is not in the vocabulary, no document can contain it and the result is empty.

For example, for "women cotton kurta" the term `women` appears in 13,807 documents, `cotton` in 19,918 and `kurta` in 1,196. Only **575 documents** contain the three terms, so the AND filter discards more than half of the documents that contain the rarest term.

2.3 Test queries (Task 2)

Following the hint of the statement, we chose the query words by popularity. Table 1 shows the ten terms with the highest document frequency (df), their collection frequency (cf, total occurrences) and their idf (Section 2.4). These terms are useless as query words: `cloth` appears in 27,722 of the 28,080 documents (98.7%), so its idf is 0.02 and it cannot separate one product from another. Most of them come from the category and from the field names of product_details (`color`, `fabric`, `pattern`, `wash`), which appear in almost every product.

Table 1. The ten terms with the highest document frequency.

Term	df	cf	idf
cloth	27,722	31,084	0.02
accessori	27,189	29,204	0.05
color	26,862	37,498	0.06
fabric	26,450	57,370	0.09
pattern	25,783	26,595	0.12
care	22,642	23,746	0.31
wear	20,971	31,691	0.42
wash	20,528	26,831	0.45
cotton	19,918	34,079	0.49
suitabl	19,309	19,507	0.54

We therefore looked at popular words that describe a product, a garment type or a target customer (Table 2). They range from very common (`men`, `women`, with idf close to 1) to rarer and more discriminative terms (`run`, `sneaker`, `shoe`, with idf above 5). Mixing both kinds in each query lets us test whether the ranking relies on the informative terms.

Table 2. Product words used to build the test queries.

Term	df	cf	idf
cotton	19,918	34,079	0.49
shirt	16,770	50,038	0.74
neck	15,190	57,363	0.89
men	14,250	28,248	0.98
women	13,807	28,041	1.02
white	3,937	6,626	2.83
polo	3,054	11,489	3.20
winter	2,868	3,287	3.29
tshirt	2,648	4,009	3.41
hood	2,161	3,600	3.70
kurta	1,196	4,202	4.55
jacket	1,110	3,338	4.66
run	792	916	5.15
sneaker	780	803	5.17
shoe	526	1,346	5.74

Table 3. The five test queries. Identifiers 1 and 2 are reserved for the validation queries.

ID	Query	Terms after pre-processing	Retrieved (AND)	Why we chose it
3	women cotton kurta	women, cotton, kurta	575	Popular ethnic product with gender and fabric.
4	men running shoes	men, run, shoe	76	Footwear, a small part of the corpus (Men's Footwear has 920 products).
5	polo neck tshirt men	polo, neck, tshirt, men	203	Very popular garment; tests the spelling "tshirt" against "T-Shirt".
6	winter hooded jacket men	winter, hood, jacket, men	292	Tests an attribute stored in product_details (Hooded).
7	white sneakers women	white, sneaker, women	76	Combines a colour with a rare product type.

2.4 Ranking with TF-IDF (Task 3)

We use the vector space model: the query and every document are vectors with one weight per term, and documents are ranked by the cosine of the angle between both vectors. The weights follow the TF-IDF scheme with a logarithmic term frequency, in base 2 as in the theory sessions. The same formulas are applied to documents and queries.

Term frequency weight. A term that appears ten times does not make a document ten times more relevant, so the raw count tf(t, d) is dampened with a logarithm (tf = 1 gives 1, tf = 2 gives 2, tf = 4 gives 3):

$$\text{tf_w}(t, d) = 1 + \log_2 \text{tf}(t, d) \quad \text{if tf}(t, d) > 0, \text{and 0 otherwise} \quad (1)$$

Inverse document frequency. N is the number of documents (28,080) and df(t) the number of documents that contain t. Rare terms get a high idf and very common terms an idf close to 0:

$$\text{idf}(t) = \log_2(N / \text{df}(t)) \quad (2)$$

TF-IDF weight of a term in a document d or in the query q:

$$w(t, d) = \text{tf_w}(t, d) \cdot \text{idf}(t) \qquad w(t, q) = \text{tf_w}(t, q) \cdot \text{idf}(t) \quad (3)$$

Score. The cosine similarity divides the dot product by the length (norm) of both vectors, so that a long document does not win just because it contains many terms:

$$\text{score}(q, d) = \Sigma_{t \in q}\, w(t, q) \cdot w(t, d) \;/\; (|q| \cdot |d|), \quad |d| = \sqrt{(\Sigma_{t \in d}\, w(t, d)^2)} \quad (4)$$

The code splits these formulas into four functions:

Function	What it does	Formula
`tf_weight(tf)`	Returns 1 + log2(tf).	(1)
`create_tfidf_index(docs)`	Builds the index, computes the idf of every term and the norm |d| of every document once, before any query.	(2), (3), |d| in (4)
`score_tfidf(...)`	Weights the query terms (their tf in the query is counted with Counter), computes |q| and the cosine of every candidate document.	(3), (4)
`search_in_corpus(query, ...)`	Runs build_terms, the AND filter and score_tfidf, and sorts by decreasing score. Ties are broken by pid so that every run returns the same ranking.	-

Worked example. For "women cotton kurta" every term appears once in the query, so its weight equals its idf: `kurta` gets log2(28080 / 1196) = 4.553, `women` gets 1.024 and `cotton` 0.495. The query norm is √(4.553² + 1.024² + 0.495²) = 4.693, and `kurta` alone accounts for 94% of the squared length of the query vector. The ranking is therefore driven by the rare term, which is the intended behaviour of idf.

Table 4 shows the first three results of each test query. Queries 3 and 5 start with relevant products. The first result of Query 4 is a shoe for women, whose score (0.599) is almost the same as that of the men's shoes below it (0.598). Queries 6 and 7 start with non-relevant products; the reasons are analysed in Section 3.6.

Table 4. Top three results of each test query (relevance as judged in Section 3.3).

Query	Rank	Score	Title	Brand	Relevant
3	1	0.522	Women Solid Cotton Blend Straight Kurta (Black)	Purple Sta	Yes
3	2	0.505	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	Yes
3	3	0.505	Women Kurta and Pyjama Set Pure Cotton	SATDEVANGIKHADIBHAND	Yes
4	1	0.599	Running Shoes For Women (White)	Vector	No
4	2	0.598	Running Shoes For Men (Multicolor)	Vector	Yes
4	3	0.598	Running Shoes For Men (Multicolor)	Vector	Yes
5	1	0.544	Self Design Men Polo Neck Blue T-Shirt	ATHL	Yes
5	2	0.544	Self Design Men Polo Neck Dark Blue T-Shirt	ATHL	Yes
5	3	0.542	Solid Men Polo Neck Black T-Shirt	Pu	Yes
6	1	0.684	Full Sleeve Printed Men Jacket	ECKO Unl	No
6	2	0.683	Full Sleeve Solid Men Jacket	ECKO Unl	No
6	3	0.682	Full Sleeve Solid Men Jacket	ECKO Unl	No
7	1	0.342	Printed Women Round Neck White T-Shirt	Free Authori	No
7	2	0.307	Women Slim Fit Solid Casual Shirt	Solid Styl	No
7	3	0.294	Solid Women Round Neck White, Blue T-Shirt	Fairdea	No

3. Evaluation

3.1 Evaluation metrics (Task 1)

All metrics are in `myapp/search/evaluation.py` and follow the definitions of Practice Session 2. Each function receives the true labels `y_true` (1 relevant, 0 not relevant) and the scores `y_score` of the engine. The helper `rank_labels` sorts the labels by decreasing score with a stable sort, which is the first step of every metric. In the formulas, rel_i is the label at rank i, K the cut-off and R the total number of relevant documents. As a running example we use the ranked labels 0, 1, 0, 1, 1 with R = 3; these cases are checked with assert statements in the notebook.

Precision@K (`precision_at_k`) is the share of the first K results that are relevant, that is, what the user finds useful on the first page. It ignores the order inside the top K. Example: P@5 = 3/5 = 0.600.

$$\mathrm{P@K} = (1 / \mathrm{K}) \cdot \Sigma_{i=1..K}\, \mathrm{rel}_i \quad (5)$$

Recall@K (`recall_at_k`) is the share of all relevant documents that appear in the top K. The function has a parameter `total_relevant` so that R also counts the relevant documents the AND filter never returned; without it, recall would be overestimated. Its maximum value is K/R. Example: R@5 = 3/3 = 1.000.

$$\mathrm{R@K} = (1 / \mathrm{R}) \cdot \Sigma_{i=1..K}\, \mathrm{rel}_i \quad (6)$$

F1@K (`f1_at_k`) is the harmonic mean of both; it is only high when precision and recall are both high. Example: F1@5 = 2 · 0.6 · 1 / 1.6 = 0.750.

$$\mathrm{F1@K} = 2 \cdot \mathrm{P@K} \cdot \mathrm{R@K} \,/\, (\,\mathrm{P@K} + \mathrm{R@K}\,) \quad (7)$$

Average Precision@K (`avg_precision_at_k`) adds the precision at each position where a relevant document appears, so relevant documents placed early weigh more. As in Practice Session 2, it divides by R_K, the number of relevant documents found in the top K. Example: AP@5 = (1/2 + 2/4 + 3/5) / 3 = 0.533.

$$\mathrm{AP@K} = (1 \,/\, \mathrm{R_K}) \cdot \Sigma_{\mathrm{i}=1..\mathrm{K}}\, \mathrm{P@i} \cdot \mathrm{rel_i} \quad (8)$$

Mean Average Precision (`map_at_k`) is the mean AP over the set of queries Q, a single number for the ranking quality of the whole system.

$$\mathrm{MAP@K} = (1 \,/\, |\mathrm{Q}|) \cdot \Sigma_{\mathrm{q} \in \mathrm{Q}}\, \mathrm{AP@K(q)} \quad (9)$$

Reciprocal Rank and MRR (`rr_at_k`, `mrr_at_k`) only look at the position of the first relevant document (found with np.argmax). They model a user who needs just one good answer. Example: RR = 1/2 = 0.500.

$$\mathrm{RR@K} = 1 \,/\, \mathrm{rank_{first}} \ \text{ if } \mathrm{rank_{first}} \le \mathrm{K}, \text{ and } 0 \text{ otherwise} \quad (10)$$

$$\mathrm{MRR@K} = (1 \,/\, |\mathrm{Q}|) \cdot \Sigma_{\mathrm{q} \in \mathrm{Q}}\, \mathrm{RR@K(q)} \quad (11)$$

NDCG@K (`dcg_at_k`, `ndcg_at_k`) discounts the gain of each document logarithmically with its position and divides by the DCG of the ideal ranking (IDCG), so the result is between 0 and 1. We use formula 7 of the practice, with gain 2^rel − 1, which equals the label when relevance is binary. The parameter `ideal_labels` builds the ideal ranking with all relevant documents, including those not retrieved. Example: DCG@5 = 1/log2(3) + 1/log2(5) + 1/log2(6) = 1.449, IDCG@5 = 1 + 1/log2(3) + 1/log2(4) = 2.131, NDCG@5 = 0.680.

$$\mathrm{DCG@K} = \Sigma_{\mathrm{i}=1..\mathrm{K}}\, (\, 2^{\mathrm{rel_i}} - 1\,) \,/\, \log_2(\,\mathrm{i} + 1\,) \qquad \mathrm{NDCG@K} = \mathrm{DCG@K} \,/\, \mathrm{IDCG@K} \quad (12)$$

How the rankings are evaluated. The notebook function `evaluate_query` computes all metrics of one query for K = 5, 10 and 20, and `evaluate` adds MAP and MRR over a set of queries. For the validation queries only the 40 documents of `validation_labels.csv` are judged, so only those documents enter the ranking, in the order given by our engine. For our test queries every document of the corpus has a label (Section 3.3), so the whole result list is evaluated. In both cases R counts the relevant documents that the AND filter did not return.

3.2 Results on the validation queries (Task 2)

Metric	Q1, K=5	Q1, K=10	Q1, K=20	Q2, K=5	Q2, K=10	Q2, K=20
P	1.000	1.000	0.650	0.800	0.600	0.300
R	0.385	0.769	1.000	0.400	0.600	0.600
F1	0.556	0.870	0.788	0.533	0.600	0.400
AP	1.000	1.000	0.980	1.000	0.948	0.948
RR	1.000	1.000	1.000	1.000	1.000	1.000
NDCG	1.000	1.000	0.995	0.869	0.716	0.716

K	MAP	MRR
5	1.000	1.000
10	0.974	1.000
20	0.964	1.000

3.3 Ground truth for the test queries (Task 3a)

To compute recall correctly we need every relevant document of the corpus, also those that the engine misses, and judging 28,080 documents by hand for five queries is not feasible. We therefore wrote down each information need as an explicit rule and applied it to the whole collection (`RELEVANCE_RULES` in the notebook). The rules use the title and the structured product_details, which describe the product itself. The description is not used, because it often mentions other products ("pair it with sneakers"). We then read the top 20 results of each query to check that the rule agreed with our own judgment.

Table 5. Relevance criteria. All other documents get the label 0.

ID	Query	A document is relevant (1) if...	Relevant in corpus
3	women cotton kurta	the title contains "women" and "kurta", and the Fabric detail contains cotton (a kurta alone or in a set).	322
4	men running shoes	the title contains "running shoes" and "for men".	23
5	polo neck tshirt men	the title describes a men's polo neck T-shirt, however "T-shirt" is written.	1,326
6	winter hooded jacket men	it is a men's jacket of the sub_category Winter Wear and its Hooded detail is Yes.	49
7	white sneakers women	the title contains "sneakers for women" and the Color detail contains white.	3

The labels are saved in `test_labels.csv`, with the same format as `validation_labels.csv` (title, pid, query_id, labels). It contains the 1,769 judged rows: every relevant document plus the non-relevant documents of each top 20. The number of relevant documents changes a lot between queries, from 3 to 1,326, which matters when reading recall-based metrics.

3.4 Results on the test queries

Metric	Q3	Q4	Q5	Q6	Q7
P@5	1.000	0.800	1.000	0.000	0.000
R@5	0.016	0.174	0.004	0.000	0.000
F1@5	0.031	0.286	0.008	0.000	0.000
AP@5	1.000	0.679	1.000	0.000	0.000
RR@5	1.000	0.500	1.000	0.000	0.000
NDCG@5	1.000	0.661	1.000	0.000	0.000
P@10	1.000	0.600	1.000	0.100	0.000
R@10	0.031	0.261	0.008	0.020	0.000
F1@10	0.060	0.364	0.015	0.034	0.000
AP@10	1.000	0.703	1.000	0.100	0.000
RR@10	1.000	0.500	1.000	0.100	0.000
NDCG@10	1.000	0.573	1.000	0.064	0.000
P@20	1.000	0.500	1.000	0.200	0.000
R@20	0.062	0.435	0.015	0.082	0.000

Metric	Q3	Q4	Q5	Q6	Q7
F1@20	0.117	0.465	0.030	0.116	0.000
AP@20	1.000	0.642	1.000	0.167	0.000
RR@20	1.000	0.500	1.000	0.100	0.000
NDCG@20	1.000	0.512	1.000	0.147	0.000

Table 6. Metrics of the five test queries, and MAP and MRR over the five queries.

K	MAP	MRR
5	0.536	0.500
10	0.561	0.520
20	0.562	0.520

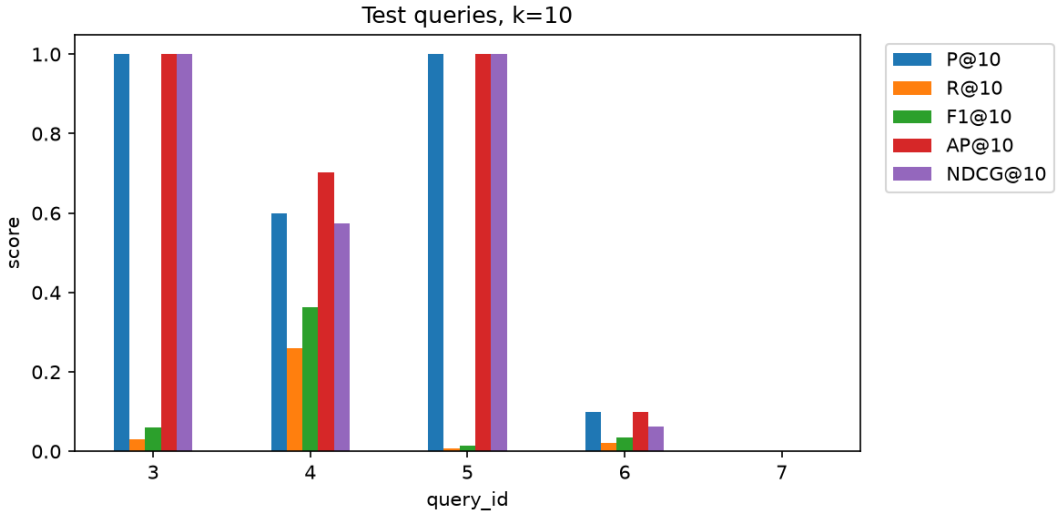


Figure 1. P, R, F1, AP and NDCG at K = 10 for each test query.

Figure 1 shows three very different behaviours. **Queries 3 and 5** have P@10, AP@10 and NDCG@10 equal to 1.000: the first ten results are all relevant. Their R@10 (0.031 and 0.008) and F1@10 (0.060 and 0.015) are almost zero, but this is not a failure of the ranking: with 322 and 1,326 relevant documents, ten results can find at most 10/322 = 0.031 and 10/1326 = 0.008 of them, which are exactly the values obtained. Recall at a small K is uninformative when a query has hundreds of relevant documents.

Query 4 is the only one where all five bars are of medium height. P@10 = 0.600 means that 6 of the first 10 results are relevant. AP@10 (0.703) is higher than P@10 because the relevant documents are concentrated at ranks 2 to 6. NDCG@10 (0.573) is lower than AP@10: NDCG compares with an ideal ranking of ten relevant documents, and it is penalised by the non-relevant documents at ranks 1, 7, 8 and 10, especially the one at rank 1, whose discount is the smallest. R@10 = 0.261 is the highest recall at K = 10, since there are only 23 relevant documents.

Query 6 has values close to zero (P@10 = 0.100, AP@10 = 0.100, NDCG@10 = 0.064): only one of the first ten results is relevant and it is at rank 10. **Query 7** has no bars at all: no relevant document is in the top 10.

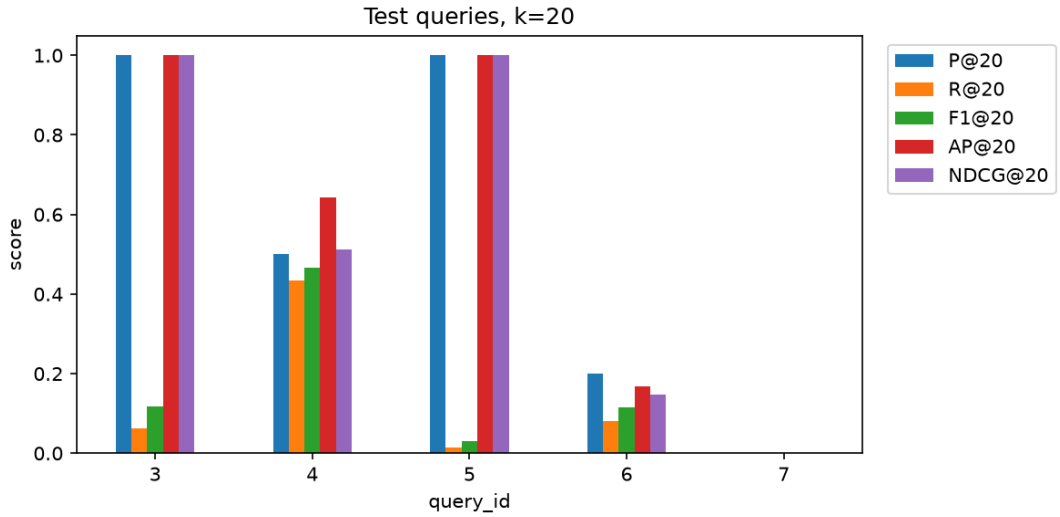


Figure 2. The same metrics at K = 20.

Figure 2 shows how the metrics change when the user looks at twice as many results. For Queries 3 and 5 precision stays at 1.000 for 20 results, and recall doubles (0.062 and 0.015), still limited by K/R. For Query 4 precision falls from 0.600 to 0.500 (only 4 of results 11 to 20 are relevant), while recall rises from 0.261 to 0.435, so F1 increases from 0.364 to 0.465, the best F1 of the five queries. AP@20 (0.642) and NDCG@20 (0.512) decrease because the new relevant documents arrive late. Query 6 improves slightly (P@20 = 0.200, AP@20 = 0.167) because three more relevant jackets appear at ranks 13, 14 and 20. Query 7 remains at 0: its first relevant document is at rank 22, just outside the cut-off.

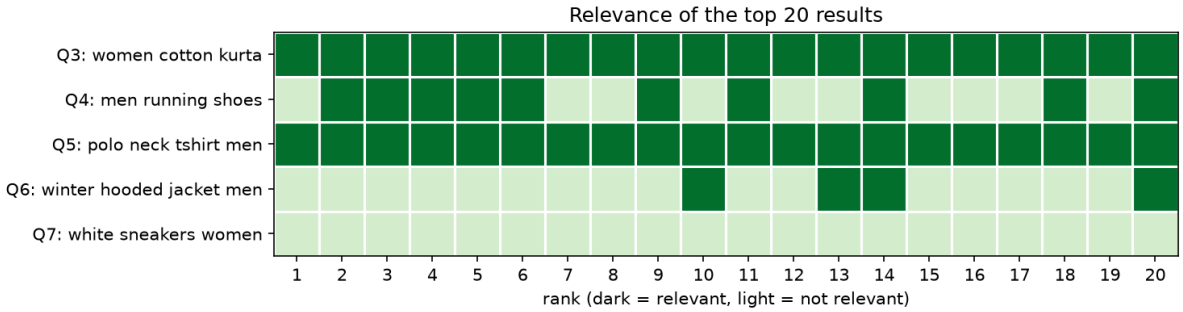


Figure 3. Relevance of each of the top 20 positions (dark: relevant, light: not relevant).

Figure 3 shows the information that the metrics summarise, position by position. The two fully dark rows (Queries 3 and 5) explain why every precision-based metric is 1.000 for them. In Query 4 the relevant positions are 2, 3, 4, 5, 6, 9, 11, 14, 18, 20: 10 of 20. Of the 10 non-relevant ones, 9 are shoes for women and one is a pack of men's socks whose description mentions running shoes, so the problem of this query is gender, not the product type. In Query 6 only positions 10, 13, 14, 20 are relevant: 16 of the top 20 are men's jackets without a hood. The row of Query 7 is completely light: the 20 results are T-shirts and shirts for women, and the 3 white sneakers are at ranks 22, 52 and 58 of 76. This view also explains the difference between RR and AP: RR is 0.500 for Query 4 because only rank 1 is wrong, and 0.100 for Query 6 because the first dark cell is the tenth.

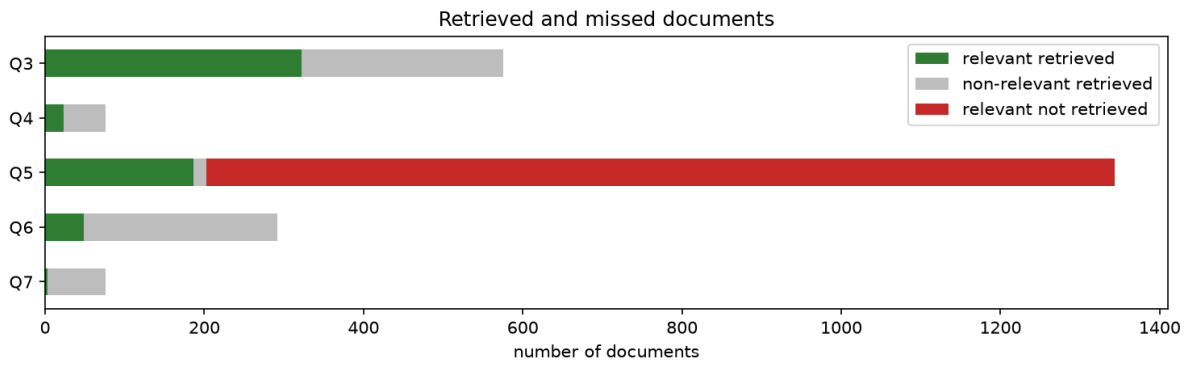


Figure 4. Documents retrieved by the AND filter (relevant and non-relevant) and relevant documents it misses.

Figure 4 looks at the complete result list instead of the top K. It separates two different problems. For Queries 3, 4, 6 and 7 there is no red bar: the AND filter retrieves **all** the relevant documents (recall over the whole list is 1.0), so their low metrics are a ranking problem. The share of relevant documents in the list is 56% for Query 3 (322 of 575), 30% for Query 4 (23 of 76), 17% for Query 6 (49 of 292) and only 4% for Query 7 (3 of 76). In Query 3, 242 of the 253 non-relevant documents are dhotis and ethnic dresses: they contain `kurta` only because their sub_category is "Kurtas, Ethnic Sets and Bottoms".

Query 5 is the opposite case. Almost everything it retrieves is relevant (186 of 203, 92%), which is why its precision is perfect, but the red bar shows 1,140 relevant T-shirts that are never returned: its recall over the whole list is only 186/1326 = 14%. This is a retrieval problem of the AND filter combined with the pre-processing (Section 3.6, problem 4), and it is invisible in Figures 1 to 3, which only show the top 20.

3.5 Comments on the evaluation metrics (Task 3b)

Table 7. What each metric measures.

Metric	Information it gives	What it ignores
P@K	Share of useful results on the first page seen by the user.	Order inside the top K; relevant documents outside it. It divides by K even if fewer documents are retrieved.
R@K	Share of all relevant documents found in the top K.	Order. Its maximum is K/R, so it is low whenever many documents are relevant.
F1@K	Balance between P@K and R@K; high only if both are high.	Order. It inherits the K/R limit of recall.
AP@K	Whether the relevant documents of one query are at the top.	With the practice definition, relevant documents never retrieved do not lower it.
MAP	One number for the ranking quality of the system.	Differences between queries: a very good query can hide a very bad one.
RR / MRR	How early the first relevant document appears.	Everything after the first relevant document.
NDCG@K	Distance to the ideal ranking, with a logarithmic discount by position; designed for graded relevance.	With binary labels it cannot separate a perfect match from a partial one.

Validation queries. Query 1 obtains P@10 = 1.000 and R@20 = 1.000: the first ten of the judged documents are relevant and the 13 relevant documents are almost entirely ahead of the 7 non-relevant ones (AP@20 = 0.980). This looks excellent, but it only means that the judged documents are well

ordered among themselves. In the full ranking of our engine they appear between ranks 149 and 500 of 500, so the first page shows documents that nobody judged.

Query 2 shows why several metrics are needed. AP@10 = 0.948 and RR = 1.000 suggest an almost perfect system, while R@20 = 0.600 and NDCG@20 = 0.716 show that 4 of the 10 relevant documents were never retrieved. Only 8 of the 20 judged documents pass the AND filter, which also explains why P@20 drops to 0.300. AP cannot see the missing documents with the definition of the practice, while NDCG can, because its ideal ranking contains all of them. Three of the four missing documents are titled "Women" (for example "Slim Women Blue Jeans") and lack the term `men`, the fourth lacks `slim`.

Test queries. MAP@10 = 0.561 and MRR@10 = 0.520 are averages of very different behaviours: two queries with perfect precision (3 and 5), one medium query (4) and two failures (6 and 7). Neither number describes any single query, which is why the per-query tables and figures are necessary. MAP also hides the recall problem of Query 5, since its AP is 1.000. Comparing with the validation set, which has MAP@10 = 0.974, would be misleading: the validation metrics only rank 40 hand-picked documents, while the test metrics rank the full result lists.

3.6 Problems of the search system and proposed solutions (Task 3c)

1. All fields have the same weight (ranking quality, field types). A term in the title counts like a term in a long description. In Query 7, T-shirts and shirts whose description mentions sneakers are ranked above the three real white sneakers: only 3 of the 76 retrieved documents have "sneaker" in the title and none is in the top 20.

> *Proposed solution:* Index the fields separately, as proposed in Part 1, and use a field-weighted score such as BM25F, giving more weight to the title and the product type.

2. Field names and values are indexed as plain words (field types, indexing). product_details is flattened into text such as "Hooded No", so the term `hood` also matches jackets without a hood. In Query 6, 181 of the 292 retrieved documents have Hooded = No, against 68 with Hooded = Yes, and 16 of the top 20 have no hood.

> *Proposed solution:* Keep attributes such as Hooded, Color or Fabric as structured fields used as filters, and map query words to them ("hooded" becomes Hooded = Yes).

3. The category text adds misleading terms (retrieval accuracy). The sub_category "Men's Footwear" also contains women's shoes, so all of them receive the term `men`: in Query 4, 39 of the 76 retrieved documents and 9 of the top 20 are for women, including the first result. In the same way, "Kurtas, Ethnic Sets and Bottoms" adds `kurta` to 242 dhotis and ethnic dresses in Query 3.

> *Proposed solution:* Index category terms in a separate low-weight field, detect gender in the query and filter by the Ideal For detail.

4. The pre-processing breaks "T-Shirt" (pre-processing, query formulation). "t-shirt" becomes `shirt`, because the single letter is dropped, while "tshirt" stays as a different term. 12,247 documents have "T-Shirt" in the title, but only 2,648 contain `tshirt`. In Query 5, 1,140 of the 1,326 relevant documents are lost, and the query "t-shirt" would also match ordinary shirts. "hooded" and "hoodie" also become different terms (`hood` and `hoodi`).

> *Proposed solution:* Normalise compound words before tokenizing ("t-shirt" and "tshirt" to one term) and add a synonym dictionary for the fashion domain, applied at indexing or query time.

5. The AND filter is too strict (retrieval accuracy, query formulation). A single missing word removes a document. In validation Query 2, 4 of the 10 relevant documents are lost this way.

Proposed solution: Keep AND as a first pass and fall back to a softer match when there are few results, for example requiring a minimum share of the query terms, or rank with OR so that documents matching more terms score higher.

6. Titles and URLs disagree on gender (data quality). In 12,764 documents the title says "Women" while the product URL says "men", for example JEAF8CHSVCP5GUH9, titled "Slim Women Blue Jeans" with the URL gas-slim-men-blue-jeans. That document is labelled relevant for "men slim jeans blue" but is not retrieved. Our own labels rely on the title and Ideal For, so they inherit this noise.

Proposed solution: Compare title, Ideal For and URL during pre-processing and flag contradictions before indexing.

7. Near-duplicates fill the first page (ranking quality). In Query 3 the top 20 contain only 2 different products (same title and brand), mostly "Women Kurta and Pyjama Set Pure Cotton" in different colours, with only 7 different scores. P@20 = 1.000 hides that the user sees the same product many times.

Proposed solution: Group colour variants of the same product, or diversify the results, for example with Maximal Marginal Relevance.

8. The score only uses the text (ranking quality). Rating, price and stock are not part of the score: 8 of the 100 top-20 results of the test queries are out of stock.

Proposed solution: Remove or demote out-of-stock products and combine the text score with the average rating, or learn the combination with Learning to Rank.

9. The evaluation has its own limits. The validation set only judges 40 documents, far from the first results of the engine, and documents with identical titles get different labels (for example "Slim Men Blue Jeans" appears with label 1 and with label 0). Our labels come from explicit rules, on five queries, and are binary.

Proposed solution: Judge the top results of the engine for every new version, use more queries and graded relevance, so that NDCG can use its full scale.

4. Conclusions and implications for the next phases

- **The AND filter and the ranking fail in different queries.** For four of the five test queries the filter keeps all relevant documents and the problem is the order (Figure 4). For Query 5 the order is perfect but 86% of the relevant documents are never retrieved. A single metric such as MAP does not show this; precision, recall and the full result list must be read together.
- **Most errors come from how fields are turned into text.** Category names (`men`, `kurta`) and attribute names (`hood`) produce matches that do not describe the product. Field-weighted ranking and structured attribute filters are the main improvements for Part 3.
- **Pre-processing must handle domain spellings**, such as "t-shirt" and "tshirt", before any ranking model can help.
- **Non-textual signals** (stock, rating) and result diversification should enter the final score.

5. Reproducibility

The README in `project_progress/part_2` gives step-by-step instructions. The notebook `part2.ipynb` reads `processed_docs.json` from Part 1, builds the index, runs every query and writes `test_labels.csv` and the figures in `evaluation_output/`. Rankings are deterministic, because ties are broken by pid.

6. AI usage statement

For this part the lecturer allowed the use of AI. We used Claude (Anthropic) to implement the indexing, ranking and evaluation code, to propose the test queries and the relevance rules, to run the experiments and to draft this report. We reviewed the code, the relevance labels and the results, and we revised the text of the report.